

Lexical Analyzer Program in C:

Aim:

To write a C program that performs lexical analysis of a C statement by identifying and classifying tokens such as keywords, identifiers, constants, arithmetic operators, relational/assignment operators, and special symbols.

Algorithm:

1. Read the C statement as input.
2. Store the list of C keywords.
3. Scan the input character by character.
4. Identify and classify keywords, identifiers, constants, operators, and special symbols.
5. Display each token with its corresponding category.

Pseudocode:

BEGIN

1. Read a C statement as input.
2. Store all C keywords in a list.
3. Scan the input statement character by character.
4. If the character is a white space,
Skip it.
5. If the character is a letter or '_',
Read all letters, digits, and '_'.
If the word is a C keyword,
Display it as Keyword.
Else
Display it as Identifier.
6. If the character is a digit,
Read the complete number.
Display it as Constant.
7. If the character is +, -, *, /, or %,
Display it as Arithmetic Operator.
8. If the character is =, <, or >,
Check the next character.
If it forms >=, <=, or ==,

Display it as Relational Operator.
Else
Display it as Operator.

9. If the character is (,), {, }, [,], :, or ,,
Display it as Special Symbol.
10. If the character does not match any category,
Display it as Unknown Symbol.
11. Continue scanning until the end of the input.

END

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[])
{
    char keywords[][20] = {
        "auto", "break", "case", "char", "const",
        "continue", "default", "do", "double",
        "else", "enum", "extern", "float", "for",
        "goto", "if", "int", "long", "register",
        "return", "short", "signed", "sizeof",
        "static", "struct", "switch", "typedef",
        "union", "unsigned", "void", "volatile",
        "while"
    };

    int i;

    for (i = 0; i < 32; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main()
{
    char input[1000];
```

```

int i = 0;

printf("Enter a C statement:\n");
fgets(input, sizeof(input), stdin);

while (input[i] != '\0')
{
    /* Ignore white spaces */
    if (isspace(input[i]))
    {
        i++;
        continue;
    }

    /* Identifier or Keyword */
    if (isalpha(input[i]) || input[i] == '_')
    {
        char word[50];
        int j = 0;

        while (isalnum(input[i]) || input[i] == '_')
        {
            word[j++] = input[i++];
        }

        word[j] = '\0';

        if (isKeyword(word))
            printf("%s\t\tKeyword\n", word);
        else
            printf("%s\t\tIdentifier\n", word);
    }

    /* Number */
    else if (isdigit(input[i]))
    {
        char number[50];
        int j = 0;

        while (isdigit(input[i]) || input[i] == '.')
        {
            number[j++] = input[i++];
        }

        number[j] = '\0';

        printf("%s\t\tConstant\n", number);
    }
}

```

```

}

/* Arithmetic Operators */
else if (input[i] == '+' || input[i] == '-' ||
        input[i] == '*' || input[i] == '/' ||
        input[i] == '%')
{
    printf("%c\t\tArithmetic Operator\n", input[i]);
    i++;
}

/* Relational / Assignment Operators */
else if (input[i] == '=' || input[i] == '<' ||
        input[i] == '>')
{
    if (input[i + 1] == '=')
    {
        printf("%c%c\t\tRelational Operator\n",
            input[i], input[i + 1]);
        i += 2;
    }
    else
    {
        printf("%c\t\tOperator\n", input[i]);
        i++;
    }
}

/* Special Symbols */
else if (input[i] == '(' || input[i] == ')' ||
        input[i] == '{' || input[i] == '}' ||
        input[i] == '[' || input[i] == ']' ||
        input[i] == ';' || input[i] == ',')
{
    printf("%c\t\tSpecial Symbol\n", input[i]);
    i++;
}

/* Unknown Symbol */
else
{
    printf("%c\t\tUnknown Symbol\n", input[i]);
    i++;
}
}
return 0;
}

```

Input:

10+b

Ouput:

Enter a C statement:

10+b

10 Constant

+ Arithmetic Operator

b Identifier

Result:

The C program successfully performs lexical analysis by identifying and classifying tokens such as keywords, identifiers, constants, operators, and special symbols.